

GRAPH THEORY – GROUP HOMEWORK

Minimum Spanning Tree

Informatics Engineering – Institut Teknologi Sepuluh Nopember

Group: 3

Members:

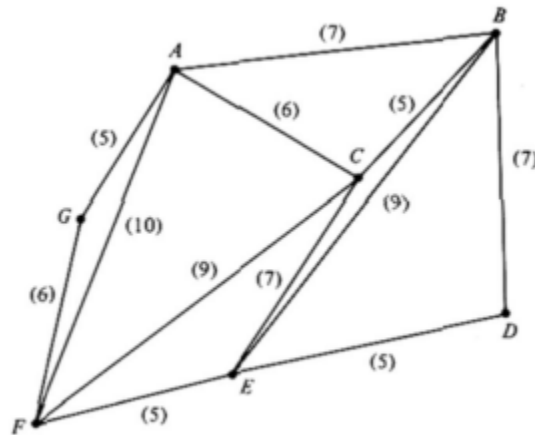
1. Dzulfiqar Rafi'ussunnah - 5025251011
2. Padhang Abiyu Fikri - 5025251014
3. Aditya Lingga Mardika - 5025251158
4. Muhammad Faris Alfarrel - 5025251002

Course: Graph Theory

1. Introduction

The purpose of this homework is to find the Minimum Spanning Tree (MST) of the given graph network. The MST connects all nodes while minimizing the total edge weight and avoiding cycles. We implement Prim algorithm, Kruskal algorithm, and another MST algorithm to find the MST. We also simulate a node or edge failure to see how the algorithm adapts to the changes in the network

1.1 Problem Statement



Graph Sample

1.2 Objective

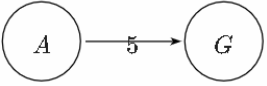
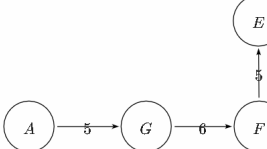
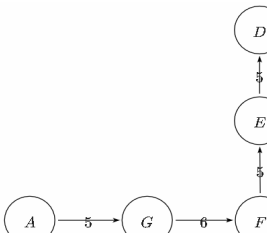
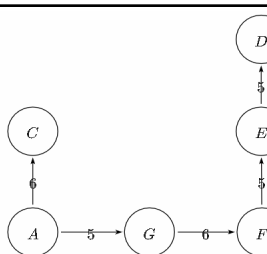
- Implement Prim's algorithm to find the MST.
- Implement Kruskal's algorithm to find the MST.
- Implement another MST algorithm that was not explained in the lecture slides.
- Simulate a failed node or edge and explain how the algorithm adapts.

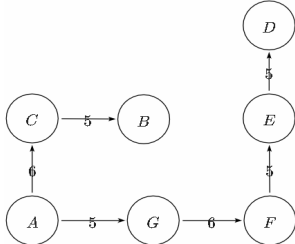
2. Prim's Algorithm

2.1 Implementation

Prim's algorithm was implemented using Python. The implementation starts from a selected vertex and maintains a visited set to track which vertices are already part of the spanning tree. At each iteration, the algorithm scans all edges connecting a vertex inside the tree to a vertex outside the tree, and manually compares their weights to find the smallest one. The edge with the minimum weight is then added to the MST, and its outside vertex is marked as visited, expanding the tree, this process repeats until all vertices have been included in the tree.

2.2 Step-by-Step Process

Step	Selected Edge	Weight	Graph
1	A-G	5	
2	G-F	6	
3	F-E	5	
4	E-D	5	
5	A-C	6	

6	C-B	5	
---	-----	---	---

Total MST Cost: [32]

2.3 Execution and Result

Selected edges for the Minimum Spanning Tree (Prim's Algorithm):

A-G = 5

G-F = 6

F-E = 5

E-D = 5

A-C = 6

C-B = 5

Total MST cost: 32

Selected edges for the Minimum Spanning Tree (Prim's Algorithm):

A-G = 5

A-C = 6

C-B = 5

G-F = 6

F-E = 5

E-D = 5

Total MST cost: 32

3. Kruskal's Algorithm

3.1 Algorithm Overview

Algorithm used to find the minimum spanning tree of a weighted graph. First, all edges are sorted from the smallest weight to the largest. Then, each edge is added to the MST if it does not create a cycle. If it creates a cycle, the edge is skipped. This process continues until all Nodes are connected and the MST has $N - 1$ edges.

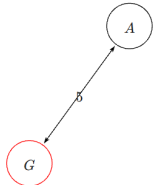
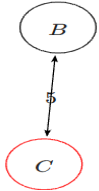
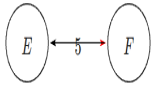
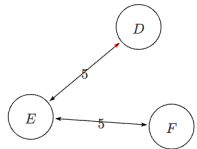
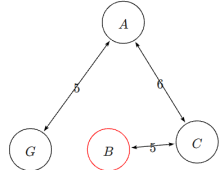
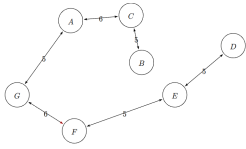
3.2 Edge List and Sorting

No.	Edge	Weight	Status
1	A-G	5	Selected
2	C-B	5	Selected
3	E-F	5	Selected
4	E-D	5	Selected
5	A-C	6	Selected
6	G-F	6	Selected
7	A-B	7	Not Processed
8	C-E	7	Not Processed
9	B-D	7	Not Processed
10	C-F	9	Not Processed
11	B-E	9	Not Processed
12	A-F	10	Not Processed

3.3 Step-by-Step Process

[Explain each edge considered by Kruskal. Clearly indicate whether the edge is ACCEPTED or SKIPPED. If skipped, explain that it creates a cycle.]

Step	Edge	Weight	Action	Cycle?	Explanation	Graph
------	------	--------	--------	--------	-------------	-------

1	A-G	5	ACCEPT	No	Connects two different components	
2	C-B	5	ACCEPT	No	Connects two different components	
3	E-F	5	ACCEPT	No	Connects two different components	
4	E-D	5	ACCEPT	No	Connects two different components	
5	A-C	6	ACCEPT	No	Connects two different components	
6	G-F	6	ACCEPT	No	Connects two different components	
7	A-B	7	Not Processed	No	Not Processed	
8	C-E	7	Not Processed	No	Not Processed	
9	B-D	7	Not Processed	No	Not Processed	
10	C-F	9	Not Processed	No	Not Processed	
11	B-E	9	Not Processed	No	Not Processed	
12	A-F	10	Not Processed	No	Not Processed	

3.4 Implementation

Kruskal's algorithm was implemented using python. The implementation uses the Union find data structure, consisting of find() and union() functions, to manage connected components and prevent cycles. The edges are sorted according to their weight and an edge is added to the MST only when it connects two different components

3.5 Execution and Result

The program successfully generated a Minimum Spanning Tree containing six edges. The selected edges were A-G, C-B, E-F, E-D, A-C, and G-F. The total MST cost obtained was **32**.

```
▼ ↩ ⏏ ⚙ 🖨
Add edge: A-G, weight = 5
Add edge: C-B, weight = 5
Add edge: E-F, weight = 5
Add edge: E-D, weight = 5
Add edge: A-C, weight = 6
Add edge: G-F, weight = 6

Minimum Spanning Tree:
A - G : 5
C - B : 5
E - F : 5
E - D : 5
A - C : 6
G - F : 6

Total MST Cost: 32
```

3.6 Final MST

Selected Edge	Weight	Cumulative Cost
A-G	5	5
C-B	5	10
E-F	5	15
E-D	5	20
A-C	6	26
G-F	6	32

Total MST Cost: 32

4. Borůvka's Algorithm

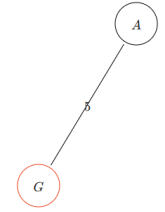
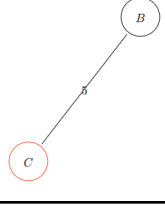
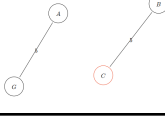
4.1 Algorithm Overview

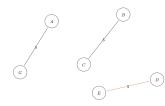
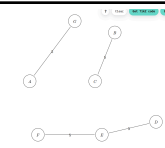
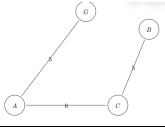
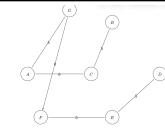
[Borůvka's Algorithm is a greedy algorithm used to find a Minimum Spanning Tree (MST). Initially, each vertex is treated as a separate component. In each round.Each component selects its cheapest outgoing edge, then the selected edges merge components. Repeat until one component remains.]

4.2 Algorithm Procedure

1. Initialize each vertex as a separate component.
2. For every component, find its cheapest edge connecting it to another component.
3. Select the cheapest outgoing edges and merge the connected components.
4. Repeat the process until all vertices belong to one component.

4.3 Borůvka Iterations

Round	Component	Cheapest Edge	Weight	Action	Resulting Component	Graph
1	[A]	A-G	5	SELECT	[A,G]	
1	[B]	B-C	5	SELECT	[B,C]	
1	[C]	C-B	5	SKIP	[B,C]	
1	[D]	D-E	5	SELECT	[D,E]	

1	{E}	E-D	5	SKIP	[D.E]	
1	[F]	F-E	5	SELECT	[D,E,F]	
1	G	G-A	5	SKIP	[A.G]	
2	[A,G]	A-C	6	SELECT	[A,B,C,G]	
2	[B,C]	A-C	6	SKIP	[A,B,C,G]	
2	[D,E,F]	F-G	6	SELECT	[A,B,C,D,E,F,G]	

4.4 Implementation

```
def boruvka(vertices, edges):
    # Initially, every vertex is a separate component
    parent = {v: v for v in vertices}
    mst = []
    total = 0

    while len(mst) < len(vertices) - 1:
        # Store the cheapest edge for each component
        cheapest = {}

        for u, v, w in edges:
            a, b = find(parent, u), find(parent, v)

            if a == b:
                continue

            if a not in cheapest or w < cheapest[a][2]:
                cheapest[a] = (u, v, w)

            if b not in cheapest or w < cheapest[b][2]:
                cheapest[b] = (u, v, w)

        # Add the selected edges to the MST
        for u, v, w in cheapest.values():
            if union(parent, u, v):
                mst.append((u, v, w))
                total += w

    return mst, total
```

4.5 Execution Result

```
# Graph from the homework
vertices = ['A', 'B', 'C', 'D', 'E', 'F', 'G']

edges = [
    ('A','B',7), ('A','C',6), ('A','G',5),
    ('A','F',10), ('G','F',6), ('F','C',9),
    ('F','B',9), ('F','E',5), ('C','B',5),
    ('C','E',7), ('E','B',9), ('E','D',5),
    ('B','D',7)
]
```

```
PS C:\Users\ASUS\Desktop\Final practicum Strukdat> python -u "c:\Users\ASUS\Desktop\Final practicum Strukdat\Tegraf.py"
Minimum Spanning Tree:
A - G : 5
C - B : 5
F - E : 5
E - D : 5
A - C : 6
G - F : 6
Total Weight: 32
PS C:\Users\ASUS\Desktop\Final practicum Strukdat>
```

4.6 Final MST

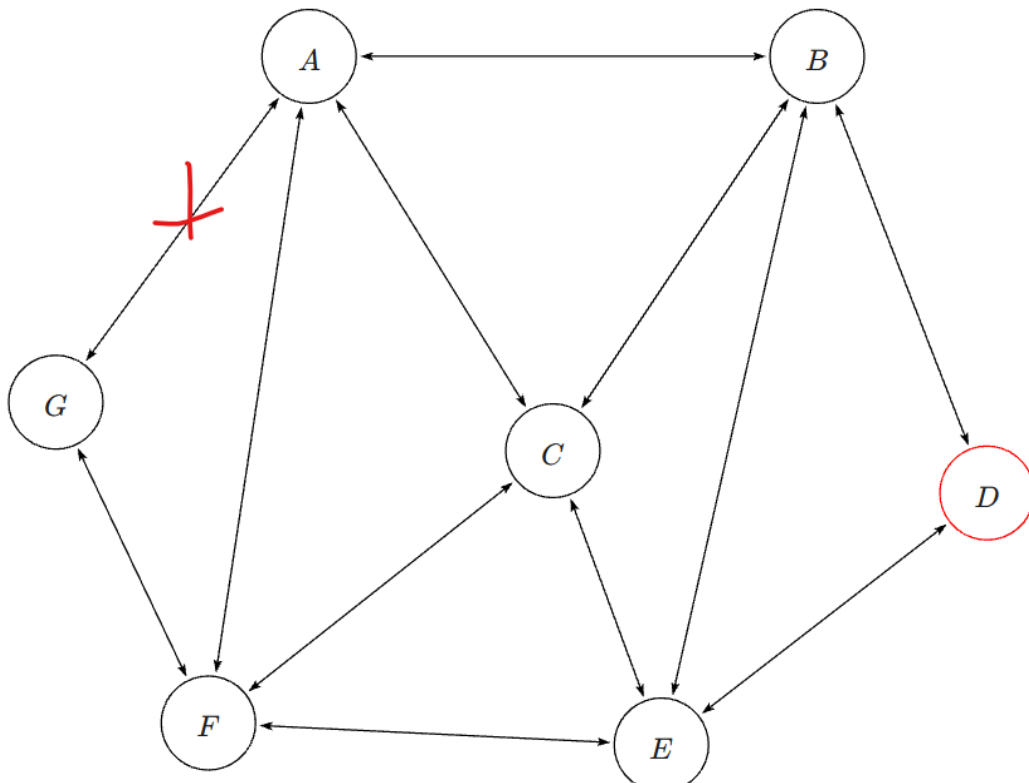
Selected Edge	Weight	Cumulative Cost
A-G	5	5
B-C	5	10
D-E	5	15
E-F	5	20
A-C	6	26
F-G	6	32

Total MST Cost: **32**

5. Node/Edge Failure Simulation

5.1 Failure Scenario

Failed element: Edge [A-G] is unavailable and cannot be traversed.



5.2 MST Before Failure

MST edges:

A-G = 5

C-B = 5

E-F = 5

E-D = 5

A-C = 6

G-F = 6

Total cost: 32

5.3 Adaptation After Failure

1. Kruskal

```
C - B : 5  
E - F : 5  
E - D : 5  
A - C : 6  
G - F : 6  
C - E : 7
```

```
Total MST Cost: 34
```

2. Prim

```
--- Edge A-G Failed (Prim) ---  
A-C = 6  
C-B = 5  
C-E = 7  
E-D = 5  
E-F = 5  
F-G = 6  
Total MST cost: 34
```

3. Boruvka

```
--- Edge A-G Failed (Borůvka) ---  
A - C : 6  
C - B : 5  
F - E : 5  
G - F : 6  
E - D : 5  
C - E : 7  
Total Weight: 34
```

5.4 Comparison

Algorithm	Condition	MST Edges	Total Cost
Kruskal	Before Failure	A-G, C-B, E-F, E-D, A-C, G-F	32
	After Failure	1. C-B, E-F, E-D, A-C, G-F, C-E 2 ETC	34
Prim's	Before Failure	A-G, G-F, F-E, E-D, A-C, C-B	32
	After Failure	A-C, C-B, B-D, D-E, E-F, F-G	34
Boruvka's	Before Failure	A-G, C-B, F-E, E-D, A-C, G-F	32
	After Failure	C-B, E-F, E-D, G-F, A-C, C-E	34

6. Algorithm Comparison

Algorithm	Approach	MST Cost	Advantages	Limitations
Prim	Similar to how a breadth-first search (BFS) expands outward, this starts at one node and grows a single tree step-by-step by always picking the cheapest connecting edge.	32	Really fast when the graph has a lot of connections (dense graphs).	Can be slower if the graph has very few connections. The network must be fully connected to work.
Kruskal	Sorts all edges from cheapest to most	32	Great for graphs with fewer edges. It can	Sorting every single edge at the beginning

	expensive, picking them one by one as long as they don't form a closed loop (a cycle).		also handle broken networks easily.	takes extra time if the graph is huge and highly connected.
Borůvka	Starts with every node isolated. In each round, every node simultaneously connects to its cheapest neighbor until all the pieces merge into one big tree.	32	It halves the number of unconnected pieces every round, making it very fast.	The code is trickier to write. If two edges have the exact same cost, it can get stuck in a loop unless you write extra rules to break the tie.

7. Conclusion

All three algorithms (Prim's, Kruskal's, and Borůvka's) successfully produced the exact same optimal Minimum Spanning Tree (MST) with a baseline cost of **32**. When simulating a network failure by severing the A-G edge, the algorithms dynamically re-routed the connection using a backup path. This increased the total MST cost to **34**, but successfully kept all nodes connected.

8. GitHub Repository

Repository: <https://github.com/ZynVans/Graph-Teory-Assignment2-Group-3>

The repository contains the implementation code, sample input/output, README, and this PDF report.

9. AI Usage Disclosure

Gpt: prompt "buatkan template DOCS tugas ini"

